

Kafka

Módulo para producir, consumir y administrar topics de Apache Kafka. Soporta autenticación SASL/SSL para conectarse a Confluent Cloud, Azure Event Hubs, AWS MSK y clusters self-hosted.

Read this in other languages: [English](#), [Português](#), [Español](#)



Como instalar este módulo

Para instalar el módulo en Rocketbot Studio, se puede hacer de dos formas:

1. Manual: **Descargar** el archivo .zip y descomprimirlo en la carpeta modules. El nombre de la carpeta debe ser el mismo al del módulo y dentro debe tener los siguientes archivos y carpetas: `_init_.py`, `package.json`, `docs`, `example` y `libs`. Si tiene abierta la aplicación, refresca el navegador para poder utilizar el nuevo modulo.
2. Automática: Al ingresar a Rocketbot Studio sobre el margen derecho encontrara la sección de **Addons**, seleccionar **Install Mods**, buscar el modulo deseado y presionar install.

Como usar este modulo

Use este modulo para producir, consumir y administrar topics de Apache Kafka desde Rocketbot. Soporta autenticacion SASL/SSL para Confluent Cloud, Azure Event Hubs, AWS MSK y clusters self-hosted.

Uso basico:

1. Ejecute Conectar a Kafka con los bootstrap servers y, si corresponde, el protocolo de seguridad y las credenciales SASL. Opcionalmente defina un identificador de Sesión para mantener varias conexiones abiertas a la vez.
2. Ejecute Probar Conexion para confirmar que Rocketbot puede llegar al cluster antes de armar el resto del flujo.

3. Use los comandos de topics (Listar Topics, Crear Topic, Eliminar Topic, Describir Topic, Obtener Offsets) para administrarlos.
4. Use Producir Mensaje / Producir Batch para publicar, y Consumir Mensajes / Commit Offsets / Cerrar Consumidor para leer. Use Cerrar Sesión Kafka al final del flujo para liberar todos los recursos de una sesión.

Ejemplos de conexión:

- Kafka local o self-hosted sin autenticación: Bootstrap servers `localhost:9092` o `broker1:9092,broker2:9092`; Protocolo de seguridad `PLAINTEXT`; deje vacíos Mecanismo SASL, Usuario y Contraseña.
- Kafka self-hosted con SASL: Bootstrap servers `broker1:9093,broker2:9093`; Protocolo de seguridad `SASL_SSL` o `SASL_PLAINTEXT` según el cluster; Mecanismo SASL `PLAIN`, `SCRAM-SHA-256` o `SCRAM-SHA-512`; Usuario y Contraseña del usuario Kafka.
- Confluent Cloud: Bootstrap servers desde la configuración del cluster, normalmente `pkc-xxxxx.region.provider.confluent.cloud:9092`; Protocolo de seguridad `SASL_SSL`; Mecanismo SASL `PLAIN`; Usuario = API key; Contraseña = API secret.
- Azure Event Hubs usando Kafka API: Bootstrap servers `<namespace>.servicebus.windows.net:9093`; Protocolo de seguridad `SASL_SSL`; Mecanismo SASL `PLAIN`; Usuario exactamente `$ConnectionString`; Contraseña = connection string completa de Event Hubs, por ejemplo `Endpoint=sb://<namespace>.servicebus.windows.net/;SharedAccessKeyName=<policy>;SharedAccessKey=<key>`. Si la connection string incluye `EntityPath`, use ese mismo nombre de Event Hub como topic Kafka.
- AWS MSK: Use la lista de brokers y el modo de autenticación provistos por el cluster. Para autenticación IAM/OAUTHBEARER este módulo no alcanza tal como está; use credenciales SCRAM o PLAIN expuestas por el cluster.

Sesiones:

- Todos los comandos reciben un identificador de Sesión. Deje vacío para usar la conexión por defecto, o defina uno para mantener varias conexiones/consumers independientes en el mismo flujo (por ejemplo, uno por cluster o por consumer group).
- Conectar a Kafka solo guarda la configuración de la conexión; todavía no crea el consumer.
- Cada sesión puede tener un consumer activo. Para usar consumers independientes con distintos topics, grupos u offsets, use sesiones diferentes.
- Cerrar Sesión Kafka cierra el producer y consumer de la sesión, y luego elimina la sesión. No confirma offsets consumidos automáticamente.

Publicar mensajes:

- Producir Mensaje envía un único mensaje (clave opcional; el valor puede ser texto plano o JSON).
- Producir Batch envía una lista JSON de mensajes en una sola llamada, por ejemplo `[{"key": "1", "value": "hola"}, {"key": "2", "value": {"total": 100}}]`. Cada ítem puede ser un valor simple o un objeto `{key, value}`. Útil cuando el robot ya tiene una lista de registros para publicar juntos.

Consumir mensajes (flujo de consumer group):

1. Ejecute Consumir Mensajes con Topicos (separados por coma), Group ID y Offset inicial. Estos valores, junto con Max poll interval, solo se aplican la primera vez que se crea el consumer para esa sesion; para cambiarlos despues, ejecute primero Cerrar Consumidor y vuelva a llamar Consumir Mensajes.
2. Procese los mensajes devueltos en el robot. Cada mensaje incluye su propio **topic**, **partition**, **offset**, **key** y **value**, asi puede distinguir de que topico vino cuando esta suscripto a mas de uno.
3. Ejecute Commit Offsets para indicarle a Kafka que los mensajes fueron procesados. Por defecto confirma todo el lote; si el robot solo proceso algunos, pase los exitosos en "Mensajes procesados" para confirmar solo esos (y los anteriores), dejando el resto para reintentar en la proxima lectura.
4. Ejecute Cerrar Consumidor al terminar el flujo, o antes de cambiar Topicos, Group ID, Offset inicial o Max poll interval en la misma sesion.

Notas importantes:

- Probar Conexion nunca corta el flujo: devuelve true o false para que el robot decida segun el resultado. Si falla, el motivo se imprime en el log de ejecucion, no en la variable de resultado.
- Commit Offsets devuelve false, sin lanzar error, cuando no habia nada nuevo que confirmar desde el ultimo commit -- esto es normal, no una falla. Si el commit en si falla (por ejemplo, el consumer fue expulsado del grupo), el comando lanza un error en vez de devolver false.
- Los valores de los mensajes se guardan y devuelven tal cual se enviaron, incluyendo espacios, saltos de linea o formato del texto original (Producir Mensaje/Batch no los recortan ni reformatean, y Consumir Mensajes los devuelve sin cambios).
- Group ID, Offset inicial y Max poll interval aplican por igual a todos los topicos de una misma llamada a Consumir Mensajes. Para usar configuracion distinta por topico, use una sesion separada para cada uno.
- Si Offset inicial queda vacio, el modulo usa por defecto el comportamiento **latest** de Kafka. Esto evita procesar mensajes viejos cuando un consumer group no tiene offset confirmado, pero tambien significa que despues de consumir mensajes sin ejecutar Commit Offsets, cerrar el consumer y crearlo de nuevo puede devolver una lista vacia en vez de entregar otra vez esos mensajes anteriores. Use **earliest** explicitamente cuando necesite leer desde el primer offset disponible para un grupo nuevo/sin commits, por ejemplo en pruebas o reprocesos controlados.
- Si Commit Offsets se ejecuta mas tarde que el Max poll interval configurado desde el ultimo Consumir Mensajes, Kafka puede haber expulsado al consumer del grupo; vuelva a ejecutar Consumir Mensajes o aumente el Max poll interval.
- Eliminar Topic es irreversible y, segun la configuracion del cluster, tambien borra los mensajes del topico. Verifique bien el nombre antes de correrlo en produccion.
- La primera vez que el modulo corre en una maquina, instala automaticamente la libreria **confluent-kafka** si no hay una version compatible ya incluida; esto requiere acceso a internet en esa primera ejecucion.

Referencias:

- <https://kafka.apache.org/documentation/>
- <https://docs.confluent.io/platform/current/clients/confluent-kafka-python/html/index.html>

Descripción de los comandos

Conectar a Kafka

Configura la conexión al clúster Kafka. Para Apache Kafka/self-hosted use broker:9092 con PLAINTEXT o su configuración SSL/SASL. Para Confluent Cloud use SASL_SSL + PLAIN, usuario=API key y contraseña=API secret. Para Azure Event Hubs por endpoint Kafka use .servicebus.windows.net:9093, SASL_SSL + PLAIN, usuario=\$ConnectionString y contraseña=connection string completa de Event Hubs. Puede usar un identificador de sesión para cambiar entre varias conexiones

Parámetros	Descripción	ejemplo
Bootstrap servers	Lista de brokers separados por coma (hostpuerto). Ejemplos localhost9092, pkc-xxxxx.region.provider.confluent.cloud9092, namespace.servicebus.windows.net9093 para Azure Event Hubs.	broker1:9092,broker2:9092
Protocolo de seguridad	Protocolo de seguridad de la conexión, por defecto PLAINTEXT	
Mecanismo SASL	Mecanismo de autenticación SASL, requerido si el protocolo usa SASL	
Usuario	Usuario SASL. En Confluent Cloud es la API key; en Azure Event Hubs con Kafka debe ser \$ConnectionString.	usuario
Contraseña	Contraseña SASL. En Confluent Cloud es el API secret; en Azure Event Hubs es la connection string completa.	secr3t_p@ss
Sesión	Identificador de la conexión, si se deja vacío se usará la conexión por defecto	Conn1
Resultado	Variable donde se almacena el resultado de la conexión	conectado

Probar Conexión

Prueba si Rocketbot puede conectarse a Kafka con la configuración indicada, devolviendo true o false; si falla, el motivo se imprime en el log de ejecución

Parámetros	Descripción	ejemplo
Sesión	Identificador de la conexión configurada con 'Conectar a Kafka'	Conn1
Timeout (segundos)	Tiempo máximo de espera de la prueba	5
Resultado	Variable con true si la conexión fue exitosa, o false en caso contrario	conexion_ok

Listar Topics

Lista los topics disponibles en Kafka. Un topic es como un canal donde se publican y leen mensajes

Parámetros	Descripción	ejemplo
Sesión	Identificador de la conexión	Conn1
Timeout (segundos)	Tiempo máximo de espera de la consulta	5

Parámetros	Descripción	ejemplo
Resultado	Variable donde se almacena la lista de topics [{topic, partitions}]	topics

Crear Topic

Crea un nuevo topic en Kafka, indicando nombre, cantidad de particiones y factor de replicación si corresponde

Parámetros	Descripción	ejemplo
Tópico	Nombre del nuevo tópico	mi-topico
Particiones	Cantidad de particiones del tópico, por defecto 1	1
Factor de replicación	Factor de replicación del tópico, por defecto 1	1
Sesión	Identificador de la conexión	Conn1
Resultado	Variable donde se almacena el resultado de la creación	resultado

Eliminar Topic

IRREVERSIBLE: elimina un topic de Kafka. Según la configuración del clúster, también borra los mensajes asociados. Verifique bien el nombre del tópico antes de correrlo en producción

Parámetros	Descripción	ejemplo
Tópico	Nombre exacto del tópico a eliminar. Esta acción no se puede deshacer	mi-topico
Sesión	Identificador de la conexión	Conn1
Resultado	Variable donde se almacena el resultado de la eliminación	resultado

Describir Topic

Muestra información detallada de un topic: particiones, líder, réplicas, ISR, estado de error y configuración. Sirve para diagnóstico

Parámetros	Descripción	ejemplo
Tópico	Nombre del tópico a describir	mi-topico
Timeout (segundos)	Tiempo máximo de espera de la consulta, por defecto 10	10
Sesión	Identificador de la conexión	Conn1
Resultado	Variable con {topic, partitions [{partition, leader, replicas, isrs, error}], config {...}}	detalle_topico

Obtener Offsets

Consulta el offset más viejo y el más nuevo disponible por cada partición de un tópico. Sirve para saber cuántos mensajes hay disponibles o desde qué punto puede leer un consumer

Parámetros	Descripción	ejemplo
Tópico	Nombre del tópico a consultar	mi-topico
Timeout (segundos)	Tiempo máximo de espera de la consulta, por defecto 10	10
Sesión	Identificador de la conexión	Conn1
Resultado	Variable con lista [{partition, earliest_offset, latest_offset, messages_available}]	offsets

Producir Mensaje

Envía un mensaje individual a un topic. Por ejemplo, publicar un JSON con datos de una factura, orden o evento

Parámetros	Descripción	ejemplo
Tópico	Nombre del tópico donde se publicará el mensaje	mi-topico
Clave	Clave del mensaje, determina la partición de destino	clave (opcional)
Mensaje	Contenido del mensaje a publicar (string o JSON), se envía tal cual se escribe, incluyendo saltos de línea	contenido del mensaje
Sesión	Identificador de la conexión	Conn1
Resultado	Variable donde se almacena el resultado del envío	resultado

Producir Lote de Mensajes

Envía varios mensajes juntos a un topic. Sirve cuando el bot tiene una lista de registros y quiere publicarlos todos en Kafka

Parámetros	Descripción	ejemplo
Tópico	Nombre del tópico donde se publicarán los mensajes	mi-topico
Mensajes (JSON)	Lista JSON de mensajes. Cada item puede ser {key, value} o un valor simple	[{"key": "1", "value": "hola"}, {"key": "2", "value": {"total": 100}}]
Sesión	Identificador de la conexión	Conn1
Resultado	Variable con {sent, failed} con la cantidad enviada y los errores	resultado

Consumir Mensajes

Crea un consumer que lee mensajes de uno o varios topics, cada uno con su propio campo 'topic'. Tiene límite de cantidad y de tiempo. El consumer (topics, group ID, offset inicial, max poll interval) se crea en la primera llamada de la sesión y se reutiliza después; ejecute 'Cerrar Consumidor' primero para cambiar esos valores

Parámetros	Descripción	ejemplo
Tópicos	Tópicos a consumir, separados por coma. Si pone más de uno, cada mensaje puede venir de cualquiera de ellos; el campo 'topic' indica de cuál	topico1,topico2
Group ID	Identificador del consumer group, compartido por todos los tópicos de 'Tópicos'. Solo se aplica al crear el consumer	mi-grupo
Offset inicial	Desde dónde empezar a leer si no hay offset guardado. Solo se aplica al crear el consumer	
Timeout (segundos)	Tiempo máximo de espera por mensaje antes de continuar	5
Máximo de mensajes	Cantidad máxima de mensajes a traer en esta ejecución	10
Max poll interval (segundos)	Tiempo máximo entre llamadas a 'Consumir Mensajes' o 'Commit Offsets' antes de que Kafka expulse al consumer por inactividad. Por defecto 1800 (30 min). Solo se aplica al crear el consumer	1800
Sesión	Identificador de la conexión	Conn1
Resultado	Variable donde se almacena la lista de mensajes recibidos. Cada mensaje incluye topic, partition, offset, key y value, así se puede distinguir de qué tópico vino cuando se consumen varios a la vez	mensajes

Commit Offsets

Confirma que los mensajes consumidos fueron procesados e indica a Kafka desde dónde continuar la próxima lectura. Por defecto confirma todo el lote de 'Consumir Mensajes'; use 'Mensajes procesados' para confirmar solo hasta el último exitoso

Parámetros	Descripción	ejemplo
Sesión	Identificador de la conexión	Conn1
Mensajes procesados (opcional)	Opcional. Lista JSON con los mensajes que el bot procesó con éxito (mismo formato que devuelve 'Consumir Mensajes'). Si se deja vacío, se confirma todo el lote leído	[{"topic": "facturas", "partition": 0, "offset": 41}]
Resultado	Variable con true si se confirmó, o false si no había nada nuevo que confirmar	resultado

Cerrar Consumidor

Cierra la sesión del consumidor y libera sus recursos. Es necesario antes de cambiar la configuración del consumer (Group ID, Offset inicial, Max poll interval) en la próxima llamada a 'Consumir Mensajes'

Parámetros	Descripción	ejemplo
Sesión	Identificador de la conexión	Conn1

Cerrar Sesión

Cierra el producer y consumer activos de la sesión indicada, libera sus recursos y elimina la sesión. No confirma offsets consumidos automáticamente

Parámetros	Descripción	ejemplo
Sesión	Identificador de la conexión	Conn1
Timeout flush (segundos)	Tiempo máximo para esperar a que el producer envíe mensajes pendientes antes de cerrar la sesión. Por defecto 10	10